

Name of the Student	MADDIRALA BALAMURALI KRISHNA
Reg. No	192325015
GitHub Link	https://github.com/krishnalsk/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 1
Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0403

SLOT :

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Duration: 90 minutes

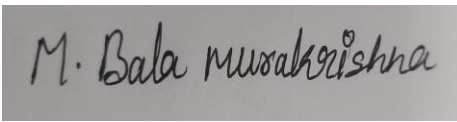
Assessment Tool Weightage: 25%

Marks: 25

Code of Conduct:

I MADDIRALA BALAMURALI KRISHNA certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



M.BALA MURALI KRISHNA

Signature of the student:

Name of the student:

Case Study Question 1: CNN for Automated Medical Image Classification (10 Marks)

Assumption: The dataset consists of grayscale/RGB chest X-ray images resized to a fixed resolution (e.g., 224×224), labelled as Normal or Disease-affected (binary classification), with a moderate-sized training set (thousands of images) that benefits from transfer learning.

1. Proposed CNN Architecture

A CNN architecture suited to this binary medical imaging task should progressively extract low-level features (edges, textures) and combine them into high-level, disease-relevant patterns (opacities, nodules, consolidations) while controlling the parameter count to avoid overfitting on a limited medical dataset. A suitable architecture is outlined below:

Layer	Configuration	Purpose
Input	224 × 224 × 1 (or 3) X-ray image	Accepts the pre-processed, normalized Xray image
Conv Block 1	Conv (32 filters, 3×3, ReLU) + Max-Pool (2×2)	Detects low-level features: edges, ribs, boundaries
Conv Block 2	Conv (64 filters, 3×3, ReLU) + Batch Norm + MaxPool (2×2)	Detects textures and simple shapes (lung fields, rib patterns)
Conv Block 3	Conv (128 filters, 3×3, ReLU) + Batch Norm + Max-Pool (2×2)	Combines features into mid-level patterns (opacities, structures)
Conv Block 4	Conv (256 filters, 3×3, ReLU) + Max-Pool (2×2)	Extracts high-level disease-specific abnormal regions
Global Average Pooling	Reduces each feature map to one value	Cuts parameters drastically vs. flattening; reduces overfitting
Fully Connected	Dense (128, ReLU) + Dropout (0.5)	Learns non-linear combinations of extracted features
Output	Dense (1, Sigmoid) or Dense (2, Softmax)	Produces the Normal / Disease-affected classification probability

For a real deployment, transfer learning from a pretrained backbone (e.g., ResNet-50 pretrained on ImageNet, fine-tuned on chest X-rays) is recommended, since medical datasets are typically small relative to the number of parameters a deep CNN needs to learn from scratch.

2. Motivation for CNNs and Role of Each Layer

Motivation: Traditional machine learning approaches for X-ray classification would need hand-crafted features (edge detectors, texture descriptors), which are difficult to design for subtle disease patterns and do not generalize well. CNNs remove this bottleneck because they automatically learn a hierarchy of visual features directly from pixel data, learning simple features (edges) in early layers and complex, disease-relevant abnormalities in deeper layers — exactly the kind of representation needed for chest X-ray interpretation.

- Convolutional layers: Slide small learnable filters (kernels) across the image to produce feature maps that highlight local patterns such as edges, opacities, and textures relevant to lung abnormalities. Each filter specializes in detecting a specific visual pattern.
- Filters (kernels): Small weight matrices (e.g., 3×3) that are convolved with the input; different filters in the same layer learn to detect different local patterns, and stacking layers lets the network combine simple patterns into complex ones.

- Pooling layers (max/average pooling): Downsample feature maps, retaining the strongest activations while reducing spatial dimensions. This provides translation invariance (a disease pattern is recognized regardless of small shifts in its position) and reduces computation and overfitting.
- Fully connected layers: Placed near the output, they take the high-level features extracted by the convolutional/pooling stack and combine them non-linearly to make the final classification decision (normal vs. disease-affected).

3. How Parameter Sharing Reduces Trainable Parameters

In a fully connected (dense) network, every input pixel is connected to every neuron in the next layer with its own independent weight. For a $224 \times 224 \times 1$ image feeding a layer of just 100 neurons, this alone requires $224 \times 224 \times 100 \approx 5.01$ million weights for a single layer.

In a convolutional layer, parameter sharing means the same filter (e.g., a $3 \times 3 \times 1$ kernel = 9 weights, plus 1 bias) is reused ("slid") across every spatial location of the image to produce one entire feature map. So a convolutional layer with 32 such filters uses only $32 \times (3 \times 3 \times 1 + 1) = 320$ parameters — regardless of whether the input image is 224×224 or 1024×1024 .

Effect: Because the same small set of weights detects a given pattern (e.g., an edge) wherever it appears in the image, parameter sharing (a) drastically reduces the number of trainable parameters compared with a fully connected layer, (b) makes the network translation-invariant, and (c) lowers the risk of overfitting — which is critical given the limited size of medical imaging datasets.

4. Regularization Techniques to Reduce Overfitting

- Dropout: Randomly deactivates a fraction of neurons (e.g., 50%) during training, forcing the network to not rely on any single neuron/feature and to learn more robust, redundant representations — reducing overfitting to noise in the small/medium X-ray dataset.
- Data augmentation: Applies realistic transformations (small rotations, horizontal flips where anatomically valid, brightness/contrast jitter, slight zoom/crop) to artificially expand the effective size and diversity of the training set, which is especially valuable for medical images where labelled data is scarce and expensive to obtain.
- Batch normalization: Normalizes the activations of each mini-batch to have stable mean and variance, which stabilizes and speeds up training, allows higher learning rates, and has a mild regularizing effect by adding a small amount of noise during training.
- Additional measures: L2 weight decay, early stopping based on validation loss, and transfer learning (freezing early pretrained layers) further help control overfitting when the labelled dataset is limited.

5. AlexNet vs. ResNet — Which Is More Appropriate?

Recommendation: ResNet (e.g., ResNet-50) is more appropriate for automated chest X-ray classification, for the following reasons:

- Feature learning: Disease patterns in X-rays (subtle opacities, small nodules) require rich, hierarchical, finegrained feature extraction; ResNet's greater depth learns more discriminative, high-level representations than AlexNet's shallower 8-layer design.
- Network depth & vanishing gradients: A deeper network is normally more expressive, but plain deep networks suffer from vanishing gradients. ResNet's residual/skip connections solve this, enabling it to train very deep networks (50+ layers) reliably, whereas AlexNet cannot be made much deeper without training instability.

- Computational requirements: ResNet is more computationally expensive than AlexNet, but for a healthcare application where diagnostic accuracy is far more important than raw speed, this extra cost is justified — especially since inference (not training) is the routine operational cost, and modern GPUs handle it efficiently.
- Classification performance: ResNet architectures (and their pretrained ImageNet weights, fine-tuned via transfer learning) consistently achieve higher accuracy on complex medical imaging benchmarks than AlexNet, making ResNet the stronger choice when misclassification (e.g., a missed disease case) carries serious real-world consequences.

Conclusion: Given the priority on diagnostic accuracy over raw inference speed, and the availability of pretrained ResNet backbones that mitigate the small-dataset problem via transfer learning, ResNet is the recommended architecture for this medical image classification system.

Case Study Question 2: CNN-Based Autonomous Vehicle Object Recognition (10 Marks)

Assumption: The system must perform real-time, multi-class object detection/classification (cars, pedestrians, traffic signs, bicycles, obstacles) from onboard camera streams, under varying lighting, viewpoint, distance, and weather conditions, with strict low-latency requirements for safety.

1. CNN Architecture Design for the Autonomous Vehicle System

Because this is a real-time, multi-object, safety-critical task, the CNN is best used as the backbone feature extractor inside a detection framework (e.g., a Faster R-CNN, SSD, or YOLO-style single-shot detector) rather than a simple single-label classifier. A suitable design:

Layer	Configuration	Purpose
Input	RGB camera frame (e.g., 416×416×3 or 608×608×3)	Accepts real-time video frames from vehicle-mounted cameras
Backbone Conv Blocks	Series of Conv + BatchNorm + ReLU + Pooling/Strided-Conv blocks (increasing filters: 32→64→128→256→512)	Extracts a hierarchy of features: edges → shapes → object parts → whole objects
Residual/Skip Connections	Identity shortcuts between blocks (ResNet-style)	Enables a deeper, more accurate backbone while keeping training stable
Multi-scale Feature Maps	Feature Pyramid Network (FPN) style fusion at multiple resolutions	Detects both small distant objects (signs) and large near objects (cars) at different scales
Detection Head	Convolutional layers predicting bounding boxes + class scores per anchor/grid cell	Localizes and classifies cars, pedestrians, signs, bicycles, obstacles simultaneously
Output	Bounding boxes with class labels and confidence scores	Final real-time recognition output used for driving decisions

2. Purpose of Each Layer and Progressive Feature Learning

- Input layer: Receives the raw camera image (pixels), typically normalized and resized to a fixed resolution for consistent processing.
- Convolution layers: Apply learnable filters across the image to detect local visual patterns; early layers detect simple low-level features such as edges, corners and colour gradients.
- Activation layers (ReLU): Introduce non-linearity after each convolution, allowing the network to model complex relationships (without which stacked convolutions would collapse into a single linear operation).

- Pooling layers: Downsample feature maps to reduce spatial size and computation while retaining the strongest, most relevant activations, and add a degree of robustness to small shifts and scale changes in object position.
- Fully connected / output layers: In a classification head, combine extracted high-level features into a final class decision; in a detection head, convolutional layers directly regress bounding-box coordinates and class probabilities per spatial location.

Progressive feature learning: In the earliest convolutional layers, filters learn low-level features — edges, colour blobs, gradients. Middle layers combine these into mid-level features — corners, wheels, limb shapes, sign borders. Deeper layers assemble these into high-level, object-specific features — a recognizable car silhouette, a pedestrian's body shape, or a stop-sign's octagonal outline — which the detection head then uses to localize and classify each object in the scene.

3. Importance of Parameter Sharing

As with the medical imaging case, each convolutional filter's weights are shared (reused) across every spatial position of the input frame. This is especially important for the autonomous driving scenario because:

- Computational efficiency: A shared 3×3 filter needs only ~10 parameters regardless of image resolution, versus millions of independent weights in a fully connected equivalent — critical for a system that must run in real time within the vehicle's onboard compute budget.
- Translation-related feature detection: A pedestrian, car, or traffic sign must be recognized correctly no matter where it appears in the frame (near the left edge, centre, or right edge, at different distances). Because the same filter scans the whole image, the network detects the same object equally well at any location — this translation invariance is essential since vehicles, pedestrians and signs constantly change position relative to the camera as the vehicle moves.
- Faster training and inference: Fewer parameters mean fewer computations per frame, which directly supports the low-latency, real-time inference required for safe driving decisions.

4. Overfitting Risk and Recommended Regularization

Overfitting is a real risk here because road-scene data can be highly repetitive in some conditions (e.g., mostly daytime, clear-weather driving) while under-representing rare but critical cases (night-time, fog, rain, unusual obstacles) — the model can memorize common patterns and fail to generalize to rare, safety-critical scenarios.

- Data augmentation: Simulate varied conditions — brightness/contrast changes, synthetic rain/fog/noise, random crops, scale changes, and horizontal flips — to expose the network to diverse lighting, weather and viewing-angle variations it must handle in the real world.
- Dropout: Applied in the fully connected/detection head layers to prevent the network from over-relying on specific feature combinations seen frequently in training data.
- Batch normalization: Stabilizes training across the very deep backbone and provides a mild regularization effect, improving generalization across varying camera/lighting conditions.
- Additional measures: Collecting/curating a diverse, balanced dataset (different times of day, weather, geographies), transfer learning from large driving datasets, and early stopping with a held-out validation set representing rare edge cases.

5. AlexNet vs. ResNet — Architectural Comparison

Aspect	AlexNet	ResNet
Depth	8 learned layers (5 conv + 3 FC)	Very deep (18/34/50/101/152 layers) via
Aspect	AlexNet	ResNet
		residual blocks
Key idea	Stacked conv + ReLU + max-pool + dropout	Skip/shortcut (identity) connections that add input to output of a block
Parameter efficiency	~60 million parameters, mostly in FC layers	Fewer parameters relative to depth; uses global average pooling instead of large FC layers
Vanishing gradient	Prone to vanishing gradients if made deeper	Solved by residual/skip connections that allow gradients to flow directly backward
Training difficulty	Easier to train (shallow) but limited representational power	Harder to train without residual connections, but skip connections make very deep training stable
Feature-learning capability	Learns good low/mid-level features; limited high-level abstraction	Learns rich hierarchical features; residual blocks let it learn identity mapping when extra layers are not needed
Computation cost	Lower FLOPs, faster inference, smaller memory	Higher FLOPs, more computation, but better accuracy per added depth
Typical accuracy	Good baseline, moderate accuracy on complex datasets	State-of-the-art accuracy on large, complex, fine-grained datasets

6. Recommended Architecture for the Autonomous Vehicle System

Recommendation: ResNet (or a ResNet-based detection backbone, e.g., ResNet + FPN within a YOLO/Faster RCNN framework) is the more suitable choice, justified as follows:

- **Accuracy:** Autonomous driving is safety-critical — misclassifying a pedestrian or missing a stop sign can have severe consequences. ResNet's deeper architecture and residual connections deliver substantially higher recognition accuracy and better generalization across the wide variety of objects, distances and conditions described in the case.
- **Computational cost:** While ResNet is more computationally expensive than AlexNet, this can be managed using efficient ResNet variants (e.g., ResNet-18/34), model compression, quantization, and dedicated onboard GPUs/AI accelerators — the accuracy gain justifies the additional but manageable cost.
- **Real-time application requirements:** With hardware acceleration and optimized/pruned ResNet backbones, real-time frame rates (e.g., 30+ FPS) needed for safe driving are achievable, while still retaining the deep, residual-connection-based feature learning that AlexNet's shallow, non-residual design cannot match.

Conclusion: ResNet's superior accuracy, robust deep feature learning, and ability to be optimized for real-time deployment make it the recommended backbone for the autonomous vehicle's CNN-based object recognition system, over the shallower and less accurate AlexNet.